

GPU Programming and Real-Time Rendering

Timothy Clarke
University of Dundee
United Kingdom
2712139@dundee.ac.uk

1. Instructions

1.1. Build & Run

The application is built using cmake. It should be run from the project's root folder.

```
$ cmake -B build
$ make -C build
$ ./assignment-01
```

1.2. Usage

Movement: **WASD** keys for movement. **R** and **F** to move up and down. Move the mouse to change camera rotation.

Torch: **T** to toggle the torch's power, **[** and **]** to tighten and widen the beam. Use the mouse **scroll wheel** to rotate through the different coloured lens filters.

Main Light: **L** switches the main light off and on. The **keypad** numbers **8**, **4**, **6**, and **2** can be used to move the main light along the x and z axis. **9** and **3** will move it up and down the y axis. **Keypad 5** will reset the light to its initial location.

Orbs: **O** pauses the animation of the orbs. **Shift+O** toggles their visibility (and their light) entirely.

Mobile: **M** pauses the rotation of the cot mobile. **P** pauses the rotation of the mobile, and the ornaments.

Animation: **Keypad +** and **-** adjusts all animation speeds.

Phong Specular Method: **F5** switches between Blinn Phong Specular and Phong Specular.

Exit: **Escape** exits the program.

1.3. Key Features

1. The cot mobile rotates continuously at a constant speed in one direction on the y-axis.
2. Ornaments rotate “naturally” in alternating directions on the y-axis. Speed is slowed at the end of a rotation, and increased again when the direction has changed.
3. Train wheels rotate continuously at a constant speed in one direction on the z-axis.
4. Primary lighting provided by five floating orbs, which move above ornaments along randomised Bezier curves.
5. Additional light source (switched off by default) provides stronger light.
6. Weak light also provided by the “adventure” torch.
7. The torch's beam may be made wider or tighter.
8. Tighter torch beams have longer reach than wider beams.
9. The torch's colour filters may be changed using the mouse scroll wheel.
10. WASD and RF (up/down) movement, with viewing angle controlled by the mouse.

2. Decision Rationale

2.1. Scene Graphs

The SceneNode class can be used to create a graph of Renderable objects. Each node may optionally have a Renderable, and zero or more child SceneNodes. An empty node is valid, yet pointless. Using a graph provides easy management of mesh construction, storage, and destruction. And provides efficient transformation and animation.

Primarily, the graph allows a transformation in a parent node, to be reflected in all descendant nodes, without setting each separately — translating, scaling, and/or rotating a parent, will provide a transformation matrix for each descendant with the same transformations. The transformations are one way, from parent to child, allowing further transformations in the children without affecting parents, cousins, etc.

Methods are provided to allow relative and absolute transformations (e.g. rotate and setRotation), and access to each transformation (e.g. getRotation).

Each SceneNode may have zero or more animations. These animations are updated after every call to animate. An animation should update the SceneNode's transformation according to its purpose, taking advantage of the transformation mechanics described above.

By using shared pointers, the SceneNode graph also allows an easy way to store all scene objects. The client may choose to keep its own shared pointer of a node for later access, or access it from the graph using the getChildren accessor.

Rendering a SceneNode does *not* render the children. Graph traversal should be done manually, to allow shader uniform values to be updated appropriately. A call to updateTransformation should be made once, to the top parent node, before the rendering loop begins. This allows any nodes marked as dirty to update their transformation matrix just once.

2.2. Triangle Arrays

My own research into OpenGL practices has led me to understand that modern GPU architectures are optimised for triangle arrays. By taking advantage of this, we no longer have to specify degenerate triangles, and only need to make one call for the whole sphere to the GPU, instead of one call per ring. This all comes at the cost of a higher number of vertices. However, in modern GPUs, space is no longer the premium that it once was.

3. Source Materials

In addition to attending lectures, I also used Learn OpenGL [1] and OpenGL Programming Guide [2] to learn in my own time.

3.1. glo/camera.*

I asked DeepSeek [3] to generate code for the lookAt method, which changes the camera's yaw and pitch according to the direction of the target location. Apart from modifying the code to fit into the Camera class, and adding a clamp, no other modification was required.

3.2. glo/mesh/shapes/cylinder.*

Although I have written this class myself, to aid in improving my usage of sin and cos, I did at times refer to how SphereMesh and

StarMesh work, which themselves are modifications of DeepSeek generated code.

3.3. glo/mesh/shapes/sphere.*

I asked DeepSeek to generate code for a sphere, using a triangle array. The code was modified to cater for:

- efficiency — the code was cleaned up, duplication removed, and value calculations minimised (for example, y only has to be calculated once per ring).
- types — the data types were changed to use OpenGL types (for example, from `float` to `GLfloat`).
- winding — winding had to be corrected.
- adaption — the relevant code was extracted to place into the SphereMesh. Variable names were modified for consistency with the current codebase, and `consts` specified as appropriate.

3.4. glo/mesh/shapes/star.*

I asked DeepSeek to generate code for several variations of cogs, as I was considering project ideas and investigating the possibility of a bronze clockwork mechanism. It had previously occurred to me that the geometric shapes, with the test wraps I was using at the time, reminded me of a child's play room. I realised that I could create a star mesh from similar code that DeepSeek had generated for one of the cogs. And so though the generated code has been heavily modified to suit the new shape, the original was generated by DeepSeek. The method `generateEdge` remains mostly intact to what was generated, though winding did need to be fixed. Overall, code was modified to cater for:

- efficiency — the code was cleaned up, duplication removed, and value calculations minimised. In particular, instead of four loops to generate the front and back faces, two of the loops were put into a method, and adapted to generate either face according to the z coordinate argument.
- types — the data types were changed to use OpenGL types (for example, from `float` to `GLfloat`).
- winding — winding had to be corrected.
- adaption — the relevant code was modified to create a star instead of a cog, and extracted to place into the StarMesh. Variable names were modified for consistency with the current codebase, and `const` specified as appropriate.

3.5. glo/scene/animators/orb.*

I asked DeepSeek to generate code for the Bezier curves. I have experience in using Bezier curves with computer art programs, but no prior knowledge of how they work. Therefore, apart from fitting the code into the animator, the Bezier curve calculation is fully generated by DeepSeek.

3.6. scene/graphs/light_properties.h

The Ogre3d wiki [4] was used for linear and quadratic values for various light ranges.

I asked DeepSeek to extrapolate linear and quadratic values from the data for ranges that I felt more desirable.

3.7. scene/torch.*

I had a discussion with DeepSeek to find suitable properties for an "adventure torch". This included colours for the plastic filters, and a colour for a bare incandescent bulb. I considered the beam width and attenuation suggestions too large, so asked for pen light properties and used those values instead.

3.8. textures/*.png

I made several prompts and refinements to Gemini AI [5] to generate images for the sphere (`animals.png`), cube (`letters-cube.png`), and star (`sun.png`). Each image was requested separately (i.e. two images for the sphere, six for the cube, and two for the star). These images

were modified as needed using Inkscape and Gimp, and added to appropriate texture wraps.

4. UML

A simplified UML diagram has been made for the source code. It can be found in the document `assignment-01-uml.pdf`.

References

- [1] J. de Vries, *Learn OpenGL: Learn Modern OpenGL Graphics Programming in a Step-by-Step Fashion*. Erscheinungsort nicht ermittelbar: Kendall & Wells, 2020.
- [2] J. M. Kessenich, G. Sellers, and D. Shreiner, *OpenGL Programming Guide: The Official Guide to Learning OpenGL, Version 4.5 with SPIR-V*, Ninth edition. Boston Columbus Indianapolis New York San Francisco Amsterdam Cape Town Dubai London Madrid Milan Munich Paris Montreal Toronto Delhi Mexico City São Paulo Sydney Hong Kong Seoul Singapore Taipei Tokyo: Addison-Wesley, 2017.
- [3] "DeepSeek." Accessed: Nov. 11, 2025. [Online]. Available: <https://chat.deepseek.com/>
- [4] "Point Light Attenuation." [Online]. Available: <http://wiki.ogre3d.org:0/-Point+Light+Attenuation>
- [5] "Google Gemini." Accessed: Nov. 11, 2025. [Online]. Available: <https://gemini.google.com/>